Agent 手冊:從挑劇本、啟動、讀觀測到下命令(DT + RIC 合併版,2026-09-23)

對應版本:DT orchestrator(agent_api,commit ab793ea)、ric-scenario-gw 0.8.21、dt-preproc 0.1.5、dt-exec-xapp 0.1.1。
實證:run 40–45(2026-09-22)—— 缺漏鄰區兩段窗、四種 UE/cell 級控制雙方對帳一致、PCI 混淆場 A/B。

```
Agent ──I-1/I-2 挑劇本──► DT orchestrator ──I-3 啟動──► RAN(cu/du/ru/ue 容器) ──E2──► RIC
Agent ◄──I-4/I-6/I-7 狀態/事件/變更── DT                                          │
Agent ◄──I-5 GET /observation──────────────────────────────── ric-scenario-gw ◄┘
Agent ──POST /control──────────────────────────────────────► ric-scenario-gw ──rc-probe──► E2 ──► RAN
```

一句話分工:**劇本與注入在 DT(I-1~I-4、I-6、I-7),觀測與控制在 RIC 閘道(I-5、/control)**。 Agent 不碰 InfluxDB、不碰 E2、不知道 meid。

第一部分:DT 端(劇本、啟動、事件、稽核)

Base: `http://<DT>:8302/api/v0.1/Run_orchestrator/Agent_api/` ,全 **POST + JSON**;回應 `{"success", "message", "data"}`。
時間: `offset_s` = 劇本秒(自起點); `ts_iso` = 對映牆鐘 UTC。倍速:走廊 cached 場標 10x、實測約 4.7x。

1. 挑劇本 —— I-1 `FieldReader/list`

`{"category": "corridor"}` (可選 `ready_only: true`)。
分類目前只有兩種在用: `corridor` (走廊,3 場)與 `other` (室內 live 測試場,1 場); `anr / intent / scale` 是預留值、尚無場域。ANR 題型全部由走廊場 + I-3 事件製造(見 §3 表)。

name	category	cell / UE	模式	事件
corridor24_00_08 / 08_16 / 16_24	corridor	3 / 18	cached(≤10x)	cell_outage, energy_saving_state, tx_power_change, relation_change, pci_change
livetest_corridor	other	4 / 18(含 1 備用 cell)	live(≤2x)	上述 + cell_addition, antenna_change

每筆附 `config_version` 、 `event_types` 、 `event_param_schema` (照它組 I-3 的 params)、 `load_profiles` (人流寫定, `ue_count` 只能往下裁)。

2. 看設定 —— I-2 `FieldReader/config`

`{"field_id": "scene-…"}` → `cells[]` (cell_id / ran_cell_id / **ncgi 字串 / nr_cell_identity 數值** / pci / arfcn / 功率 / 位置 / 天線 / 狀態)、 `e2_identity` (e2_node_id、plmn、每 cell 的 NCI 十進位與 hex)、 `son` (現行 SON 開關)、 `collection` (60 s 窗、基準 10 窗)、 `unavailable_fields` 。

- cell 識別三種寫法 I-3 都收:`"src"`、`"e03:src"`、`"208095-56bce6561"`。
- **`3-1` / `2-8` / `3-1-group` 要的數值 `target_nci` 直接取 `e2_identity.cells[].nr_cell_identity`** (比從 raw 觀測反查穩)。
- **UE 座標永遠沒有**(設計決定)。

3. 啟動 —— I-3 `ScenarioController/run` (唯一可寫)

```
{"field_id": "scene-…", "config_version": "405c36afe21f", "seed": 103, "duration_s": 1800,
 "ue_count": 18, "requester_task_id": "…",
 "son": {"anr_auto_add": false, "anr_pci_lookup": true, "anr_cgi_fail_rate": 0.0},
 "events": [{"event_id": "E1", "type": "relation_change", "at_offset_s": 60, "duration_s":
null,
             "target": "e03:src", "params": {"action": "add", "source_ncgi": "e03:src",
"target_ncgi": "e03:mid"}}]}
```

- `seed`、`duration_s` (≥720)必填; `ue_count` 只減不增; `magnitude` 收下不作用。
- **`son` 與 `events` 都是啟動時決定、整場固定**;事件在 `at_offset_s` 那個劇本秒由排程器執行, `duration_s` 到期自動還原(outage / es / cell_addition / pci_change)。**沒有跑到一半改設定的介面。**
- 回應: `run_id` (= I-4/6/7 的 run_id = **RIC 閘道的 scenarioId**)、`instance_id`、`echo` (實際生效值含 `son`、`ue_names`、`speed_x`)、`unavailable_fields`。
- 錯誤:400 驗證(逐筆 event_id 原因 + `supported_types`)、404、409(config_version 不符 / 場域正在跑)、503(RIC 閘道不在)。

事件	params	效果		
`cell_outage`	`reason?`	DU 停排程 + RU 停發射		
`energy_saving_state`	`state_after: energySaving\	notEnergySaving`	同上開關	
`tx_power_change`	`tx_power_dbm_after`	RU 功率 + PEE 輸入		
`relation_change`	`action: add\	remove\	set, source_ncgi, target_ncgi, attributes?`	CU NRT,與 xApp 同入口
`pci_change`	`pci_after` 0~1007	只改廣播 PCI 標籤(訊號不變)→ 與另一顆相同 = 混淆		
`cell_addition`	`tx_power_dbm?`	只能開備用 cell		
`antenna_change`	`tilt_deg_after`, `azimuth_deg_after`	live 限定		

ANR 題型對照

題型	I-3 設定	RIC 端會看到
缺漏鄰區	`son.anr_auto_add=false`(開場 NRT 空);或 `relation_change add` @60 s → `remove` @1200 s 造窗	量測有 PCI 但 `ncgi` 缺席、`neighbourCellRelations` 沒那條、換手做不了
錯誤/多餘鄰區	`relation_change add` 一條收不到訊號的關係	關係表有、`measurementReportAggregate` 沒有
關係旗標錯	`relation_change set` + `attributes`	`hoValidated` / 換手率異常
PCI 混淆	`pci_change` nbr → 與 mid 同 PCI(A: `anr_auto_add=true`;B:false + guard)	`servingCells` 兩顆同 PCI、該 PCI 量測率跳升、`ncgi` 覆蓋率掉到約一半、rsrp 上移
cell 停機/節能	`cell_outage` / `energy_saving_state`(+ `duration_s`)	關係還在、目標無訊號

4. 狀態 —— I-4 `RunReader/status`

`{"run_id"}` → `status`(running / done / failed / stopped)、`progress_s`、`duration_s`、`windows_available`、`events{total, pending, alive, deadline_s}`、`ue_names`。`done` = 劇本秒到 `duration_s` 由 DT 停止。

5. 事件 —— I-6 `RunReader/events`

`{"run_id", "from"?, "to"?, "types"?}`;型別 `meas_report, ho_exe_succ, ho_exe_fail, ho_prep_fail, rlf, rrc_reestab_req, cgi_resolution`。
每筆 `{type, offset_s, ts_iso, gnb_id, cell_id, ue_id, payload}`;`meas_report.payload.reported[]` 以 `pci/arfcn/rsrp/rsrq` 為準。

6. 變更稽核 —— I-7 `RunReader/config_changes`

`{"run_id", "from"?, "to"?, "origin"?}`;每筆 `{offset_s, ts_iso, origin, source, object_type, object_id, action, outcome, before, after, reason}`。

- `source` 前綴:** `agent/*` **= 你自己的 I-3 事件;** `ric/*` **= RIC/xApp 控制(** `outcome` **= ACK / FAIL / pending,** `detail.instId` **、** `detail.style` **);** `son/*` ** = gNB 自主(CGI 解析)與 RIC 命令落地(`son/ric_apply prb_quota_applied`)。
- **與 RIC 稽核對帳用 instId**:RIC `control_audit.instance_id` ↔ I-7 `detail.instId`。閘道的 `label` 不經 E2,I-7 看不到。

第二部分:RIC 端(觀測與控制)

閘道:叢集內 `http://ric-scenario-gw.ricplt:8080`,叢集外 `http://<RIC IP>:30990`。**所有資料經閘道進來、所有命令經閘道出去。**

7. 部署形態

形態	用途	部署
A. 叢集內 Deployment(推薦)	常駐自動決策(範例 dt-exec)	helm chart 放 `components/<name>/charts`, `bootstrap/versions.env` 加 `<NAME>_IMAGE`, `40-apps.sh` 加一行 helm;不需 RMR、不需註冊 appmgr
B. 叢集外程序 (LLM Agent)	實驗、腳本	直接打 NodePort;程式碼與 A 相同,只差 URL
C. 真正 E2 xApp	自己訂閱 E2	另見 RIC `xApp開發規範`,本手冊不涵蓋

紀律:動叢集前跑 `ric-deploy-gate.sh` (RAN 有 run 就不動);換映像改 `values.yaml` + `helm upgrade --reset-values`,不要 `--set image.tag=`。

8. 讀觀測 —— I-5

方法	路徑	用途
GET	`/v1/scenarios`	活著的場景清單(入口): `scenarioId` (= DT `run_id`)、`run`、`state`
GET	`/v1/scenarios/{id}`	單場景: `nodes{}`、`labels.scene_name`、`allowedStyles`、`xappNames`
GET	`/v1/scenarios/{id}/observation`	**主要讀口**。`format=aggregated` (預設,一份 = 最新 60 s 窗)/ `raw` (逐份 E2 報告,正序分頁 `page_size`、`next_token`、`from/to`)/ `both`; `run=N`; `at=<epoch ms>`
GET	`/v1/scenarios/{id}/series?db=&measurement=&run=&since=&limit=`	直接讀某 measurement(含 `dt_preproc.control_audit` 稽核表)
GET	`/v1/nodes/{meid}`	節點目前 `gen` (**下命令前必查**)

彙整格式頂層: `window{sceneTimeStart, sceneTimeEnd, reports, qualifiedForBaseline, reason}`、**`controlTargets[]` (空 = 不能下命令)**、`e2NodeInformation{servingCells[], neighbourCellRelations[]}`、`ueLevel[]{ueId, ngapId, servingCellNcgi, handoverInWindow, rsrp/sinr/throughput p50, controllable}`、`kpmIndication{cellLevel{"<量測>.<cell>": {current, baseline?, trendLast30Min}}, perNeighbourRelation[]}`、`e2MessageCopyAggregate{measurementReportAggregate[], reestablishmentInboundByPreviousPci[]}`、`unavailable_fields`。

語意:

- 一列 = 60 s 窗、時戳為窗**結束**,約 15 s 寬限後出現;場景 active 後約 90 s 才有第一窗(`404 no-observation` 不是錯)。
- `baseline` 是前 6 個合格窗中位數,約第 11 窗起才有;**你自己的命令會讓那窗 `control(n)` 不進基準**。
- `ncgi` 在彙整格式是 **cell 名**(`e03:src`),數值 NCI 在 raw 或 **DT I-2**。

- 鄰區以 **PCI@ARFCN** 為鍵;`ncgi` 只在 gNB NRT 恰好解出一顆時才附;兩顆同 PCI 同時可見 → 合併一筆、**不附**。
- `RRU.PrbTotDl` 是 ran_func 6 cell 級值、每窗最後一筆(與 DT 每秒峰值不同量)。
- `ueLevel[].ngapId` 是 `3-1` / `2-6` 的 UEID; `controllable=false` 下不了; `servingCellNcgi` 是該窗眾數, `handoverInWindow=true` 的 UE 執行時可能已不在那顆。
- `position` 永久沒有。

9. 下命令 —— `POST /v1/scenarios/{id}/control`

Header `X-Scenario-Gen: <GET /v1/nodes/{meid} 現查的 gen>`;body 共通 `{"style", "nodeId": "cu", "label": "<前綴>-<動作>-<目標>", "wait_ack": true, "wait_timeout": 8, …}`,**不帶 meid**(422)。
回應 `{"resolvedMeid", "audited", "rcProbe": {"ack": "ACK"|"FAILURE", "rmr_state", "instance_id", "outcome", "error", "detail"}}`。
判定順序:HTTP≠200 閘道拒 → `rcProbe.error` 有值沒送出 → `rmr_state≠0` RMR 沒送 → `ack=="FAILURE"` RAN 拒 → `"ACK"` 成功。閘道會擋: `422 style-required`、`403 style-not-allowed`、`400 bad-generation`、`409 scenario-not-active / ran-not-connected`。

style	類別	參數(鍵名照 rc-probe)	從哪拿
`rc-reportcgi`	唯讀	`pci`, `arfcn`	`servingCells[]` / `measurementReportAggregate[]`。混淆時 RAN 回 `cgi=null, confusion=true, results{兩顆}`
`anr-add`	ANR	`sourceCellId`, `target:{cgi, pci, arfcn, rat:"NR", plmn:"208095"}`	cell 名 `servingCells[].ncgi`;**pci/arfcn 要帶**(只帶 cgi 時 RAN 會用本地設定補)
`anr-remove`	ANR	`sourceCellId`, `targetCgi`, `reason`	`neighbourCellRelations[]`
`anr-flag`	ANR	`sourceCellId`, `targetCgi`, `flag` ∈hoBlocklist/noRemove/xnBlocklist, `op` ∈set/clear	同上
`2-6` PRB 配額	RC UE 級	`ngap_id`, `min_prb`, `max_prb`, `dedicated_prb`	`ueLevel[].ngapId`;RAN 落地後 DT I-7 會有 `prb_quota_applied`
`3-1` 換手	RC UE 級	`ngap_id`, `target_nci`(數值), `target_plmn_hex`("02f859")	`target_nci` ← DT I-2 `nr_cell_identity`(或 raw `neighbors[].ncgi` 反查)
`3-1-group`	RC cell 級	`serving_cell`, `pci`, `target_nci`, `target_plmn_hex`, `cond_arfcn?`	同上
`2-8` 波束	RC cell 級	`target_nci` + `tilt_decideg` / `beams` / `max_layers` / `txru_pct` / `beam_window` / `beam_power` 至少一個	同上
`ccc-lock` / `ccc-unlock` / `ccc-es-on` / `ccc-es-off`	CCC	`cell_id`(完整 cell 名 `e03:nbr`)	`servingCells[].ncgi`

10. 執行 xApp / Agent 內部必須有的(RIC 端踩坑清單)

1 只有一個資料入口(`/observation`);2 每個 (scenarioId, run, timestamp_ms) 只處理一次;3 `controlTargets` 空就不做;4 `404 no-observation` 要等不報錯;5 gen 送前現查;6 body 不帶 meid;7 判定看三層、`ack` 是字串;8 參數鍵名照 rc-probe(錯鍵會用預設值送出);9 目標冷卻 600 s + 每窗上限 3;10 `handoverInWindow=true` 的 UE 不拿來下 UE 級命令(或標明在賭);11 同 PCI 對到兩顆 cell → 不推目標;12 沒有的資料就不做、不從別處補(曾掃出假 NCI)。
留痕:每筆決策帶 evidence;`label` 固定前綴;提供 `/status`;要有 `DRY_RUN` 開關(只觀察的場先切)。

第三部分:一場的標準流程

1. `I-1` 挑 `category=corridor` 的場 → `I-2` 記下 `config_version`、cell 的 `ncgi` / `nr_cell_identity` / `pci` / `arfcn`、`son` 現值。
2. `I-3` 啟動(帶 `seed`、`duration_s`、`son`、`events`)→ 拿 `run_id`。
3. RIC `GET /v1/scenarios/{run_id}` 等 `state=active`;約 90 s 後 `GET …/observation` 有第一窗;基準約第 11 窗。
4. 迴圈:讀 observation → 規則 → `GET /v1/nodes/{meid}` 取 gen → `POST …/control`(帶 label)→ 看 `rcProbe.ack`。
5. 隨時 `I-4` 看進度;`I-6` 看事件;**`I-7` 看誰改了什麼**(自己的 `agent/*`、RIC 的 `ric/*`、gNB 的 `son/*`),與 RIC `control_audit` 以 `instId` 對帳。
6. `I-4 status=done` 收工;歸檔資料保留一輪(下一場啟動時清上一場之前的)。

附:已知待修(DT,2026-09-23)

- I-6 `meas_report.reported[]` 目前多帶 `cell_id`(上帝視角),將移除;`ho_*.target_ncgi` 目前是 cell 名,將對齊 I-2 的 ncgi 字串。
- I-4 進行中 `finished_at` 可能殘留上一場值,以 `status` 為準。
- 前端沒有事件注入面板;人工測試請直接打 I-3。